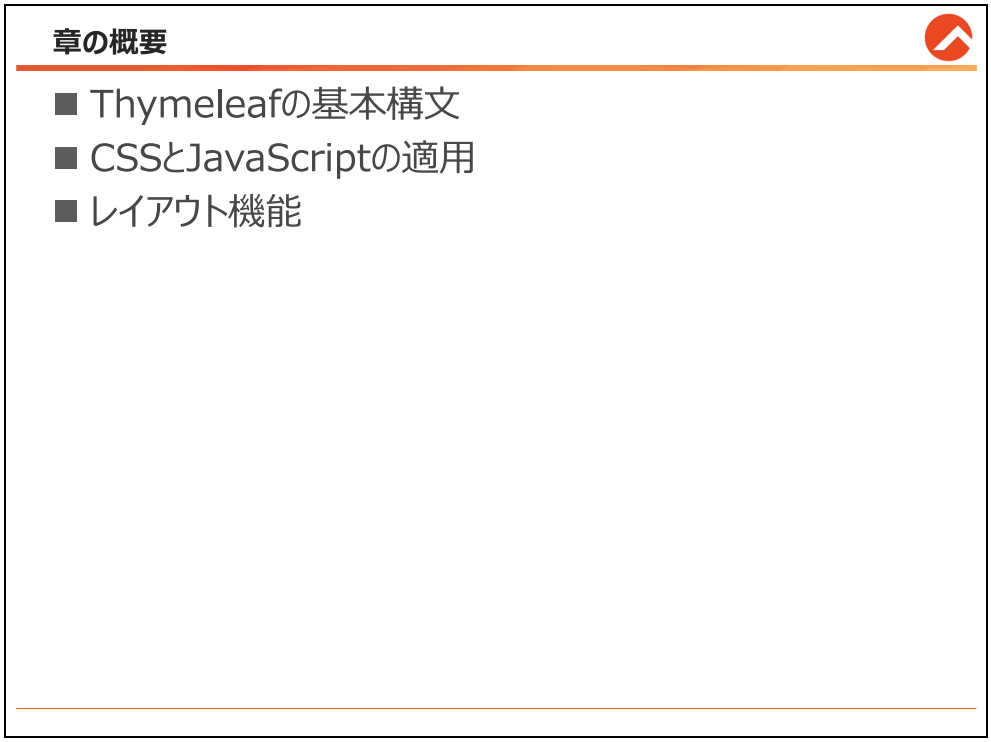
5

第5章 Thymeleaf と Web ページデザイン

章の目標

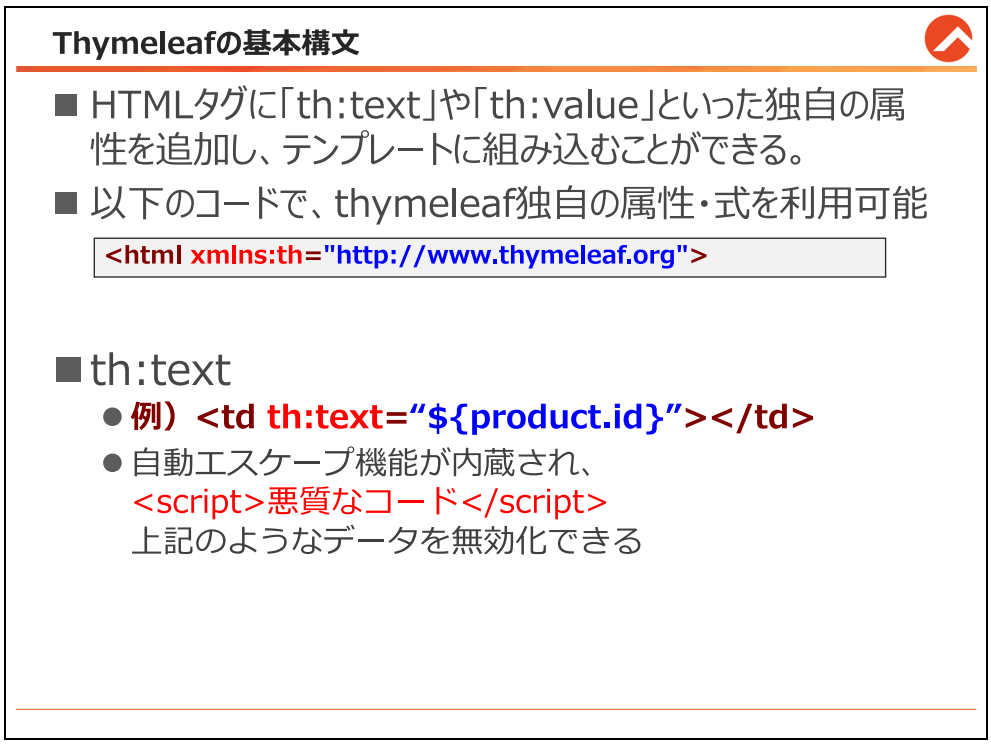
- Thymeleaf の基本構文について理解できる
- Thymeleaf から CSS や JavaScript を活用できる
- レイアウト機能について理解できる

章の概要



今まで、thymeleaf のタグについて、いくつか例文の中でご紹介しました。この章では、改めてthymeleaf の文法をメインの内容にし、CSS とJavaScript を連携する方法についてご説明します。そして、保守性と効率を高めるレイアウト機能についてもご説明します。

5.1. Thymeleaf の基本構文



Thymeleaf について

「Thymeleaf」は、JSP と同様に、Java の「テンプレートエンジン」の一種で、HTML5 形式のテンプレートを変換し、Web ブラウザでの表示を可能にするライブラリです。Thymeleaf は、コントローラーからのデータを受け取ることができますので、コンテンツの動的な変更が実現できます。Thymeleaf の主要な記述方式は以下の通りです。

- 属性記述：HTML タグに「th:text」や「th:value」といった独自の属性を追加し、テンプレートに組み込むことができる。
- 式記述：属性の値を「${}」や「@{}」の形式で表現し、動的なデータの挿入ができる。

Thymeleaf はSpring Framework との統合が容易であり、Spring アプリケーションでの使用に特に適しています。HTML の中に自然にデータを組み込むことができるため、可読性も高いです。さらに、Thymeleaf はセキュリティ面での配慮もなされており、XSS（クロスサイトスクリプティング）対策などのセキュリティ機能を内蔵しています。

Thymeleaf を使うための事前準備

HTML ファイル内で、Thymeleaf の属性や式を使うためには、以下のようなコードを記載しておく必要があります。

HTML

```HTML
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

コードの読み方

2行目

```HTML
<html xmlns:th="http://www.thymeleaf.org">
```

Thymeleaf テンプレートで使われる HTML の宣言です。この宣言により、Thymeleaf のルールが HTML ページに追加されます。それによって、Thymeleaf の特有の属性や構文が HTML タグ内で認識され、正しく処理されます。

6行目

```HTML
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

この宣言は、Thymeleaf の構文ではありませんが、Web ページがモバイルデバイスで表示される際のビューポート（表示領域）の設定に使われます。デバイスがどのような画面サイズを持っていても、Web ページはそれに適応するように表示されます。

th:text について

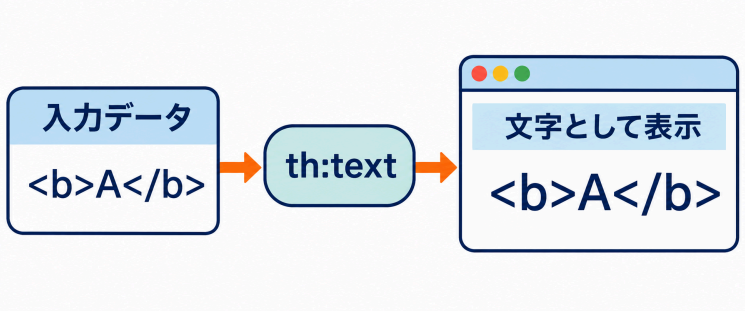
HTML

```HTML
<td th:text="${product.id}">[データがあったら、置き換えられる]</td>
<td th:text="${product.name}">[データがあったら、置き換えられる]</td>
<td th:text="${product.price}">[データがあったら、置き換えられる]
</td>
```

一番基本となる、th:text は、HTML の要素に動的なテキスト内容を挿入するために使用されます。主に、サーバー側で処理されるデータ（例えば、データベースから取得した情報やユーザ入力）をWeb ページに表示する際に用います。**<td></td> タグ内のテキストは、th:text 属性に指定された式の結果によって置き換えられます。**

th:text の特徴の一つは、クロスサイトスクリプティング（XSS）の対策です。例えば、ユーザが「**<script>悪質なJavaScript コード</script>**」 というデータを入力した場合、XSS対策が施されていないと、この悪意のあるJavaScriptコードがHTML 内に組み込まれ、不特定多数のユーザに被害を及ぼす可能性があります。

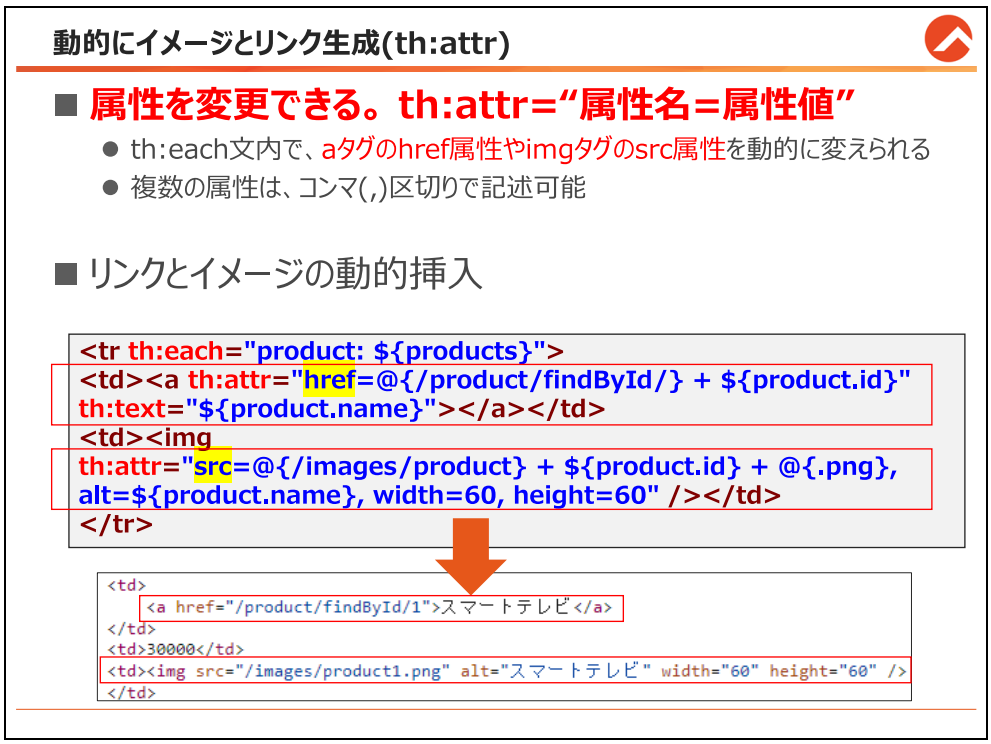
図5. 1. 1



th:textはタグに見える入力もエスケープして、HTMLの本文へ文字として出力します。

その対策として、th:text を使用すると、入力データは自動的にエスケープ処理されます。これにより、‘<’ や ‘>’ のような文字はHTML タグとしてではなく、ただの文字列として扱われます。結果として、悪意のあるJavaScript コードが実行される心配はなくなります。

5.1.1. 動的にイメージとリンク生成(th:attr)



Thymeleaf の th:attr 属性を使用することで、HTML 要素の属性を動的に変更することが可能です。データベースから取得したデータに基づいて異なる属性値を設定することができますので、**Form タグ内のid やclass 属性を動的で割り当てることができますし、画像のsrc 属性やリンクのhref 属性などを動的に変えることもできます。**

イメージの動的挿入

【振り返り】＜Spring Boot プロジェクトの構成＞

図5. 1. 2

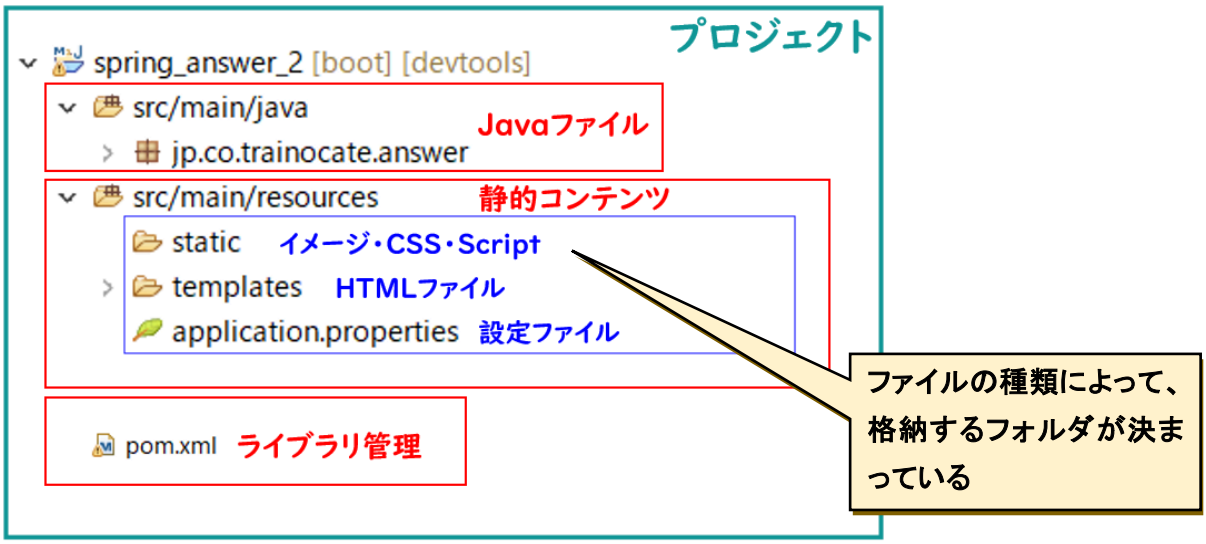
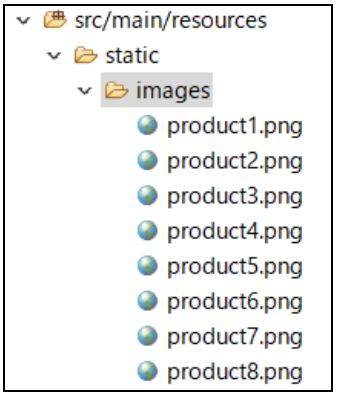


図5. 1. 3



Spring Boot 環境では、イメージのような静的ファイルをsrc/main/resources/static フォルダに格納する必要があります。今回は、予めimages フォルダに、商品ごとのイメージを格納しておきました。product の後ろの番号は、商品番号に合わせています。

HTML　…/resources/templates/product/list.html

```
1  <table>
2  <tr>
3          <th>商品番号</th>
4          <th>商品名</th>
5          <th>価格</th>
6          <th>イメージ</th>
```

HTML

```
</tr>
<tr th:each="product: ${products}">
    <td th:text="${product.id}"></td>
    <td th:text="${product.name}"></td>
    <td th:text="${product.price}"></td>
    <td><img th:attr="src=@{/images/product} + ${product.id} +
@{.png},
        alt=${product.name}, width=60, height=60" /></td>
</tr>
```

既存のfindByAll のサンプルのlist.html に、th:attr のコードを追記しました。

th:attr の構文は **th:attr="属性名=属性値"** という形で、属性値を動的に設定できます。通常、イメージを挿入する際は <img src="イメージのURL"> として src 属性にURLを指定しますが、**th:attr を使用すると <img th:attr="src=@{イメージのURL}"> のように書き換えることができます**。

この例では、product.id を用いて商品ごとに異なるイメージURL を生成しています。th:attr を使用する際に複数の属性を指定したい場合は、コンマ（,）で区切ることで設定できます。alt 属性はイメージの表示が失敗した時の代替テキストとして機能し、ここでは商品名を出力しています。また、width と height 属性を指定することでイメージの大きさを統一しています。

出力結果をみると、以下のように変換されているのがわかります。

図5. 1. 4

```
<tr>

    <td>1</td>
    <td>スマートテレビ</td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" /></td>
</tr>
<tr>

    <td>2</td>
    <td>ゲーミングモニター</td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" /></td>
</tr>
<tr>

    <td>3</td>
    <td>ノートパソコン</td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" /></td>
</tr>
```

動的URL のリンク挿入

Web システムには、以下のようにデータごとに動的なリンクがかかっていることもよくあります。

図5. 1. 5



この場合、**a タグに同じくth:attr 属性値としてhref を指定すると、動的リンクを生成**することができます。

HTML …/resources/templates/product/list.html

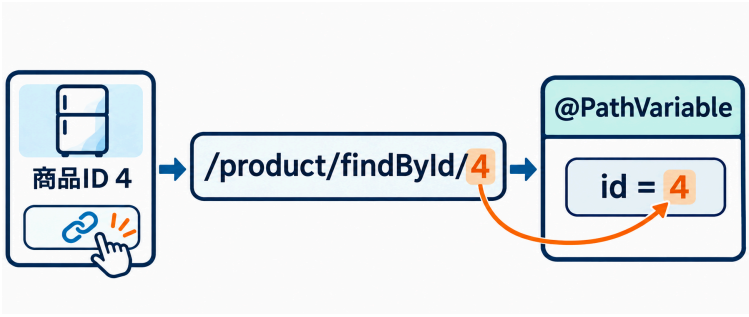
```
<tr th:each="product: ${products}">
   <td th:text="${product.id}"></td>
   <td>
       <a th:attr="href=@{/product/findById/} + ${product.id}"
       th:text="${product.name}"></a>
   </td>
   <td th:text="${product.price}"></td>
   <td><img th:attr="src=@{/images/product} + ${product.id} +
@{.png},
          alt=${product.name}, width=60, height=60" />
  </td>
</tr>
```

Java …/controller/ProductController.java

```java
@GetMapping("/findById/{id}")
public String showProductById(@PathVariable int id, Model model) {
    Product product = prodRepo.findById(id).orElse(null);
    model.addAttribute("product", product);
    return "product/detail";
}
```

動的なURL リンクによる呼び出しの場合は、URL からid を受け取る処理が必要となりますので、2 章のupdate とdelete の場合と同様に、@PathVariable アノテーションを付与する必要があります。（その場合、フォームからのid 検索は無効になります）

図5. 1. 6



動的リンクのURLに埋めた商品番号を、@PathVariableでハンドラメソッドの引数へ取り出します。

動作確認

http://localhost:8080/product/findAll

正しく指定した場合、以下のようなコードが生成されることを確認できます。

図5. 1. 7

```
<tr>
    <td>1</td>
    <td>
        <a href="/product/findById/1">スマートテレビ</a>
    </td>
    <td>30000</td>
    <td><img src="/images/product1.png" alt="スマートテレビ" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>2</td>
    <td>
        <a href="/product/findById/2">ゲーミングモニター</a>
    </td>
    <td>20000</td>
    <td><img src="/images/product2.png" alt="ゲーミングモニター" width="60" height="60" />
    </td>
</tr>
<tr>
    <td>3</td>
    <td>
        <a href="/product/findById/3">ノートパソコン</a>
    </td>
    <td>50000</td>
    <td><img src="/images/product3.png" alt="ノートパソコン" width="60" height="60" />
    </td>
</tr>
```

5.1.2. 条件式による表示制御(th:if)



①th:if とth:unless

Thymeleaf では、th:if や th:unless を活用して、条件に基づいたHTML の表示を制御することができます。これにより、データベースから取得した**データに基づいて異なる表示を行う**ことが可能です。

活用例は多岐にわたりますが、以下にその一例をご紹介します。

HTML …/resources/templates/product/list.html

```
<tr th:each="product: ${products}">
<!--省略-->

<!-- 価格が30000 より大きい場合、赤色で価格を表示 -->
<td th:if="${product.price > 30000}" th:text="${product.price}"
style="color: red;"></td>

<!-- 価格が30000 以下の場合、通常のスタイルで価格を表示 -->
<td th:unless="${product.price > 30000}"
th:text="${product.price}"></td>
```

この例では、product.price の値が30000 を超えるかどうかで、スタイルが適用された価格と、通常スタイルの価格を切り替えて表示しています。**HTML コードは2行ですが、条件に合うコードだけが表示されるようになっています。**

②その他の条件式の例

条件式による制御は、活用場面が無限に近いのですが、下記によく使われる例をご紹介します。

⑴ **削除フラグに基づくデータの表示**:

削除フラグが0 の場合はデータを表示し、1 の場合は表示しない。(テーブルにdeleteFlag の列を追加する必要があります。)

HTML

```
<div th:each="product : ${products}" th:if="${product.deleteFlag == 0}"
th:text="${product.name}"></div>
```

⑵ **管理者権限に基づく機能の表示**:

特定のユーザー（例えば、管理者）にのみ特定の機能を表示する。

(テーブルにrole の列を追加する必要があります。)

HTML

```
<div th:if="${session.user.role == 'ADMIN'}">
    <!-- 管理者専用の機能やリンクをここに表示 -->
</div>
```

⑶ **リストの要素が空かどうかのチェック**:

リストが空の場合に、「情報がありません」というメッセージを表示する。

(#lists.isEmpty(products)は、Thymeleaf で提供する変数式です。簡単にリストの検索OK/NG を判定できる構文です。)

HTML

```
<div th:if="${#lists.isEmpty(products)}">
    商品情報がありません。
</div>
<div th:unless="${#lists.isEmpty(products)}">
    <!-- 商品情報リストの内容(<table>～</table>)をここに記載 -->
</div>
```

⑷ 日付や時間に基づくコンテンツの表示：

現在の日付や時間に基づいて、特定のメッセージやコンテンツを表示する。

（**#dates.～～**は、Thymeleaf で提供する変数式です。簡単に現在の日付関連のデータを取得できる構文です。）

HTML

```
<div th:if="${#dates.dayOfWeek(...) == 2 }">
    今日は月曜特別セールです！
</div>
```

⑸ Radio ボタンのチェックの切り替え：

値が1 か2 かによって、ラジオボタンの checked 属性を動的に切り替える。

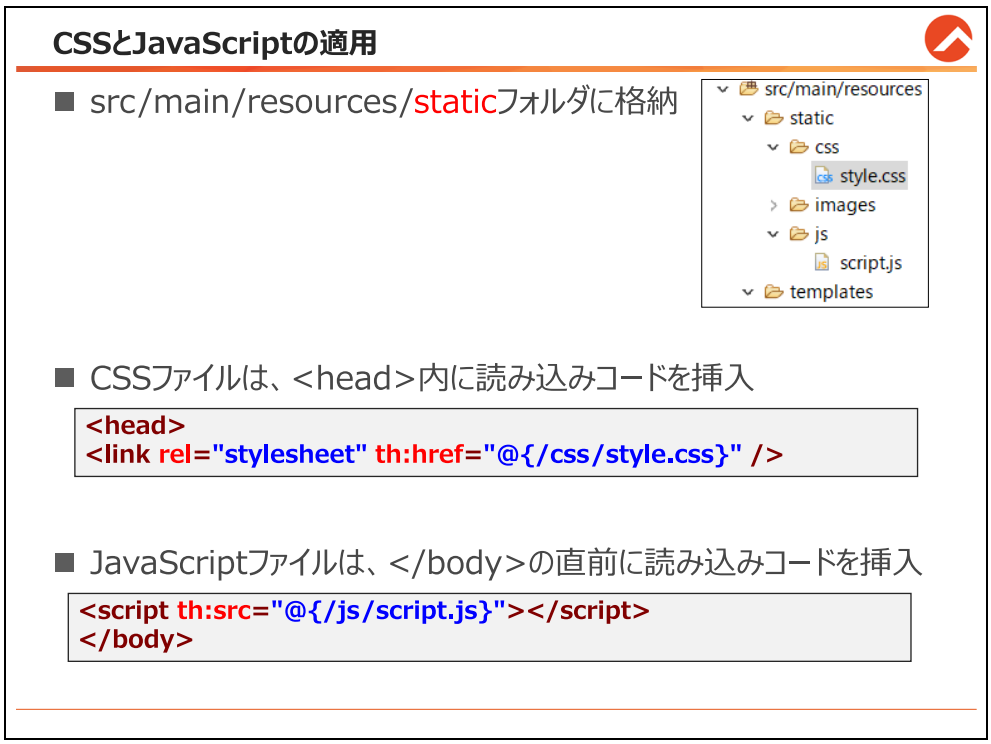
Radio ボタン以外にも、checkbox でも活用できます。

※if-unless でも書けますが、th:checked 属性を使うと、より簡略に記述できます。

HTML

```
<input type="radio" name="gender" th:value="1"
th:checked="${gender == 1}" />
<input type="radio" name="gender" th:value="2"
th:checked="${gender == 2}" />
```

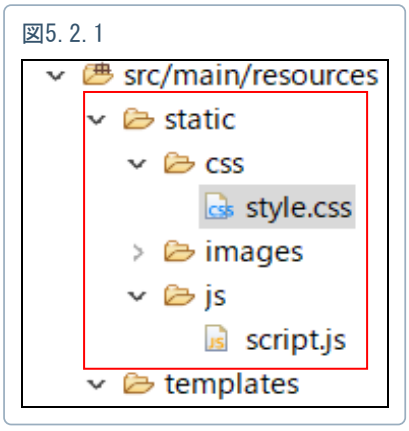
5.2. CSS と JavaScript の利用



Thymeleaf 内でCSS ファイルと、JavaScript(JS)ファイルを適用するためには、通常の方法では上手くいかないため、この節でご説明いたします。

CSS ファイルとJS ファイルの格納場所

図5. 2. 1



css ファイルとjs ファイルは、images と同様に **src/main/resources/static の中に格納しないとSpring Boot プロジェクト内で認識されませんので、必ずこの場所に格納してください。**

CSS ファイルの読み込み

HTML …/resources/templates/product2/list.html

```
1  <head>
2      <link rel="stylesheet" th:href="@{/css/style.css}" />
```

head タグの中に、th:href でファイルパスを記述すれば読み込みできます。/static までのパスは省略されます。

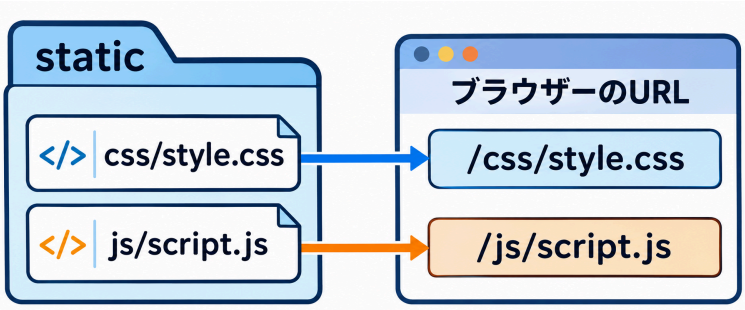
JS ファイルの読み込み

HTML …/resources/templates/product2/list.html

```
1  <script th:src="@{/js/script.js}"></script>
2  </body>
```

body の終了タグの直前の場所に、th:src でjs ファイルの場所を記述すると、読み込みできます。

図5. 2. 2



static内のCSSやJavaScriptは、ブラウザーからは/css/や/js/で始まるURLで読み込みます。

サンプルのCSS・JS ファイルのコード

CSS …/resources/static/css/style.css

```
1  @charset "UTF-8";
2  body{background-color: black;
3       color: white;}
4  h1{color: yellow;}
```

単純に背景と文字色を反転させるスタイルが記載されています。

Java …/resources/static/js/script.js

```
1  document.write("<p>このテキストはJavaScript が生成しました</p>");
```

JavaScript でHTML を生成することもできます。

動作確認

http://localhost:8080/product2/findAll

図5. 2. 3



CSS とJavaScript を連携することで、より綺麗で使いやすい画面を作ることができます。

この研修で詳細は割愛しますが、Spring Boot はREST API も簡単に作成することができますので、Ajax などで非同期処理を混ぜることで、アプリ感覚のSPA(Single Page Application)を作るのも容易です。

5.3. レイアウト機能

レイアウト機能

■ 共通のデザイン(レイアウト)ファイル

● 以下の属性をレイアウト用の<html>に追記

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
```

● 共通デザイン定義後、コンテンツが配置される場所を用意

```
<main layout:fragment="main_content">
<!-- 各ページ固有のコンテンツがここに入ります -->
</main>
```

■ コンテンツ用のHTMLファイル

● 以下の属性をコンテンツ用の<html>に追記

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
layout:decorate="~{layout}">
```

● コンテンツをlayout:fragmentのコードで囲むだけ

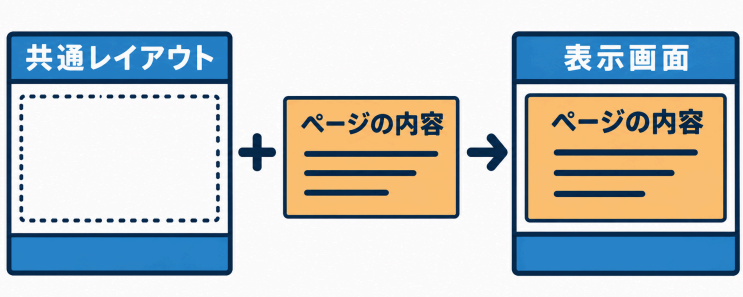
```
<main layout:fragment="main_content">  ~コンテンツ~  </main>
```

共通の要素を適用する課題について

一般的に、Web サイトは画面ごとに共通のデザインテーマを持ち、統一されたヘッダー、サイドバー、フッターを採用しています。しかし、Web サイトは数多くのhtml ファイルで構成されるものです。全てのhtml ファイルに、ヘッダー・サイドバー・フッターの要素をコピーすることを考えると、作業量も多いですし、保守性も落ちてしまいます。

Thymeleaf はこの問題を解決するための「レイアウト機能」を提供しています。**一度共通のレイアウトファイルを作成すれば、それを各ページに容易に適用することができる**ため、開発プロセスが大いに効率化されます。

図5. 3. 1



サーバー側で共通レイアウトと各ページの内容を組み合わせます。ヘッダーやフッターを再利用できます。

それでは、Thymeleaf でのレイアウトの作成方法について詳しくご説明します。

目標画面

図5. 3. 2



今回の研修では、メイン画面は画面ごと置き換えできるようにし、赤枠以外の部分を layout.html に記述します。

【事前準備】ライブラリを導入

XML　pom.xml

```xml
<dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

レイアウト機能を使うためには、関連のライブラリが必要です。追加して、maven install をしておきましょう。

【作業手順①】レイアウトファイルの作成

HTML …/resources/templates/layout.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
 xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

<head>
<meta charset="UTF-8" />
<title>Traino 商品</title>
<link rel="stylesheet" th:href="@{/css/layout.css}" />
</head>
<body>
<!-- ヘッダー -->
<header>
<h1>Traino 家電商品</h1>
</header>
<div class="flex-container">
<!-- サイドバー -->
<nav>
 <ul>
 <li><a th:href="@{/product/findAll}">全商品一覧</a></li>
 <li><a th:href="@{/product/}">商品検索</a></li>
 <li><a th:href="@{/product/insert}">商品登録</a></li>
 <li><a th:href="@{/5}">ログアウト</a></li>
 </ul>
</nav>

<!-- メインコンテンツ -->
<main layout:fragment="main_content">
 <!-- 各ページ固有のコンテンツがここに入ります -->
</main>
</div>
<!-- フッター -->
<footer>
<p>Copyright © 2027 Trainocate Spring. All rights reserved.</p>
</footer>
</body>
</html>
```

このlayout.html には、Web サイトの共通要素であるヘッダー、ナビゲーションバー、フッターが定義されています。注目すべき3 つの箇所をご説明します。

コードの読み方

3行目

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
```

レイアウトファイルを定義するためには、既存のhtml 属性に、xmlns:layout 属性を追記する必要があります。html 要素を2 回書いたわけではなく、html 要素に2 つの属性を追加していることに気をつけてください。

8行目

```
<link rel="stylesheet" th:href="@{/css/layout.css}" />
```

レイアウト専用のcss ファイルを別途作成し、読み込ませました。css ファイルの詳細は、デザインの領域ですので、章末の【参考】の部分をご覧ください。

27～29行目

```
<main layout:fragment="main_content">
 <!-- 各ページ固有のコンテンツがここに入ります -->
</main>
```

この部分のコードは、中身がありません。その理由は、ここは**メインコンテンツが配置される場所**だからです。

「layout:fragment=“識別名”」の属性をつけると、この部分に別のhtml ファイル(コンテンツ)のfragment 識別名が一致しているコードがコピーされ、**置き換えられます**。

【作業手順②】コンテンツ用のHTML の修正

HTML …/resources/templates/product/insert.html

```HTML
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
 xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
       layout:decorate="~{layout}">

       <!--省略-->

<body>
<main layout:fragment="main_content">

<!--既存のコンテンツのソースコード-->

</main>
</body>
```

修正するHTML ファイルは、商品関連のどのHTML ファイルでも構いません。

コードの読み方

3行目

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
```

レイアウトにコピーするコンテンツ側のHTML にも、同じくレイアウト関連のルールを宣言しておきます。

4行目

```
layout:decorate="~{layout}">
```

<html>にlayout:decorate 属性を追加します。~{layout} のlayout は、**templates/layout.html を指定するテンプレート名**です。

templates/product/layout.html を指定する場合は、~{product/layout} と書きます。

9～13行目

```
<main layout:fragment="main_content">

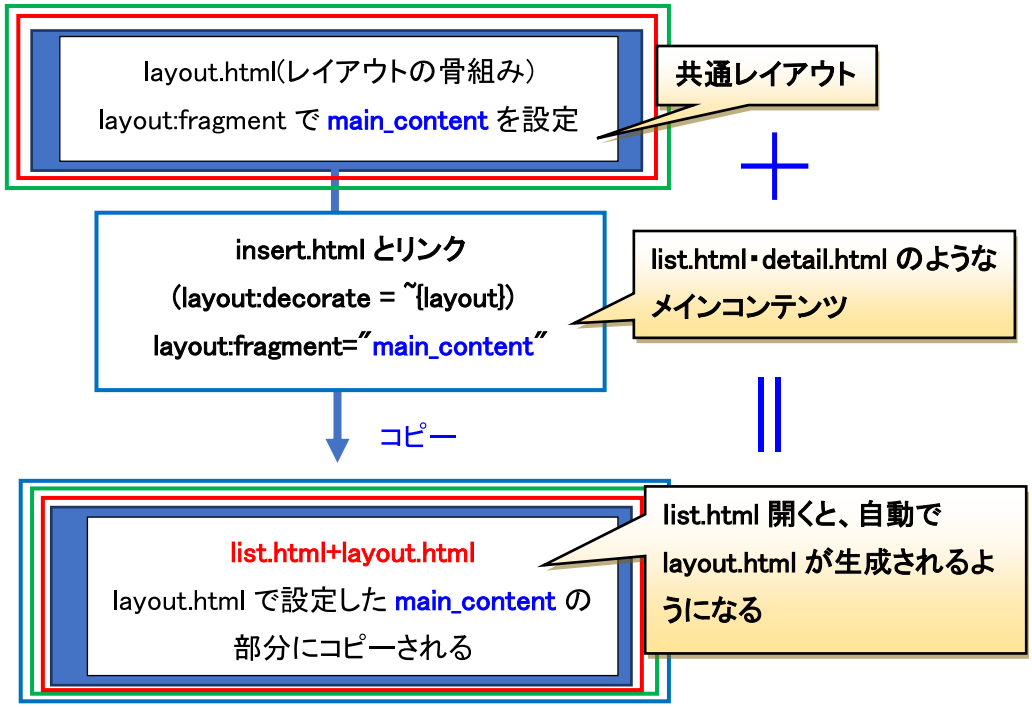
<!--既存のコンテンツのソースコード-->

</main>
```

既存の<body>内に書いていたソースコードを、全てこの要素で囲みます。先ほどの**layout.html のfragment 識別名が“main_content”でしたので、同じ値を入れます**。そうすることで、**9-13 行内の全てのHTML コードが、layout.html の28 行にコピーされ、レイアウト+コンテンツの新しいHTML が生成されます。**

insert.html へreturn して一覧画面を開くと、自動でlayout.html の共通部分まで生成された画面が表示されます。

図5. 3. 3



今回は、insert.html のみ修正しましたが、レイアウトを適用させたい全てのhtml ファイルを、同じように修正すれば、以下のようにシステム全体に同じデザインが適用されます。

図5. 3. 4


localhost:8080/product/findById/2
Traino家電商品
全商品一覧
商品検索
商品登録
ログアウト
商品詳細
商品番号 商品名 価格
2 ゲーミングモニター 20000
修正
削除
Copyright © 2027 Trainocate Spring. All rights reserved.

図5. 3. 5


localhost:8080/product/insert
Traino家電商品
全商品一覧
商品検索
商品登録
ログアウト
商品登録
商品番号：(自動割り当て)
商品名 ：
商品価格：
登録
Copyright © 2027 Trainocate Spring. All rights reserved.

【参考】layout.css のソースコード

CSS …/resources/static/css/layout.css

```css
/* 全体のベーススタイリング */
html, body {
    height: 100%;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    font-family: UD デジタル 教科書体 NP-B, sans-serif;
}

/* ヘッダーのスタイリング */
header {
        background-color: #E44D26;
    color: white;
    padding: 10px 0;
    text-align: center;
    border-bottom: 3px solid #C1351D;
}

/* サイドバーのスタイリング */
nav {
    background-color: #333; /* 黒 */
    color: white;
    padding: 15px;
    width: 200px;
    flex-shrink: 0;
}

nav ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

nav ul li a {
    color: white;
```

CSS

```
    text-decoration: none;
    padding: 10px 0;
    display: block;
    font-size: 1.2em; /* テキストサイズを大きくする */
}

nav ul li a:hover {
    background-color: #555;
}

/* メインコンテンツのスタイリング */
main {
    background-color: #fff; /* 白色の背景 */
    padding: 30px;
    overflow-y: auto; /* 必要に応じてスクロールバーを表示 */
    flex-grow: 1;
}

/* フッターのスタイリング */
footer {
    background-color: #333; /* 黒 */
    color: white;
    text-align: center;
    padding: 10px 0;
    width: 100%;
}

/* フレックスボックスのレイアウト */
.flex-container {
    display: flex;
    flex: 1;
    flex-direction: row;
}

@media (max-width: 768px) {
    .flex-container {
```

CSS 続き

```
        flex-direction: column;
    }
```

CSS

```
    nav {
        width: 100%;
    }
}

/* フッターが常に画面の最下部に表示されるようにする */
.flex-container {
    min-height: 0; /* フレックスコンテナのmin-height をリセット */
}

body > footer {
    flex-shrink: 0;
}
```

【参考】Thymeleaf標準のフラグメント

ヘッダー、メニュー、フッターは共通レイアウトに置き、**main**だけを各画面の部品に差し替えます。下のコードは、2つの方式で共通レイアウトの差し替え位置を指定する箇所です。

各画面の指定	Layout Dialect	Thymeleaf標準
共通レイアウトの指定	layout:decorate	th:replace
各画面の main	layout:fragment	th:fragment

HTML …/resources/templates/layout.html

```
<main layout:fragment="main_content">
```

HTML …/resources/templates/layout2.html

```
<main th:replace="~{::main_content}"></main>
```

商品一覧・登録・詳細に共通レイアウトを適用する

各画面で共通レイアウトを指定し、mainをmain_contentという名前で定義します。標準版の**~{::main_content}**は、共通レイアウトの呼び出し元にあるmainを選びます。

HTML …/templates/product/list.html

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layout}">
    <!-- 中略 -->
    <main layout:fragment="main_content">
        <h1>商品一覧</h1>
```

HTML …/templates/fragment_sample/list2.html

```
<html lang="ja" xmlns:th="http://www.thymeleaf.org"
      th:replace="~{layout2}">
<body>
    <main th:fragment="main_content">
        <h1>商品一覧</h1>
```

HTML …/templates/fragment_sample/insert2.html

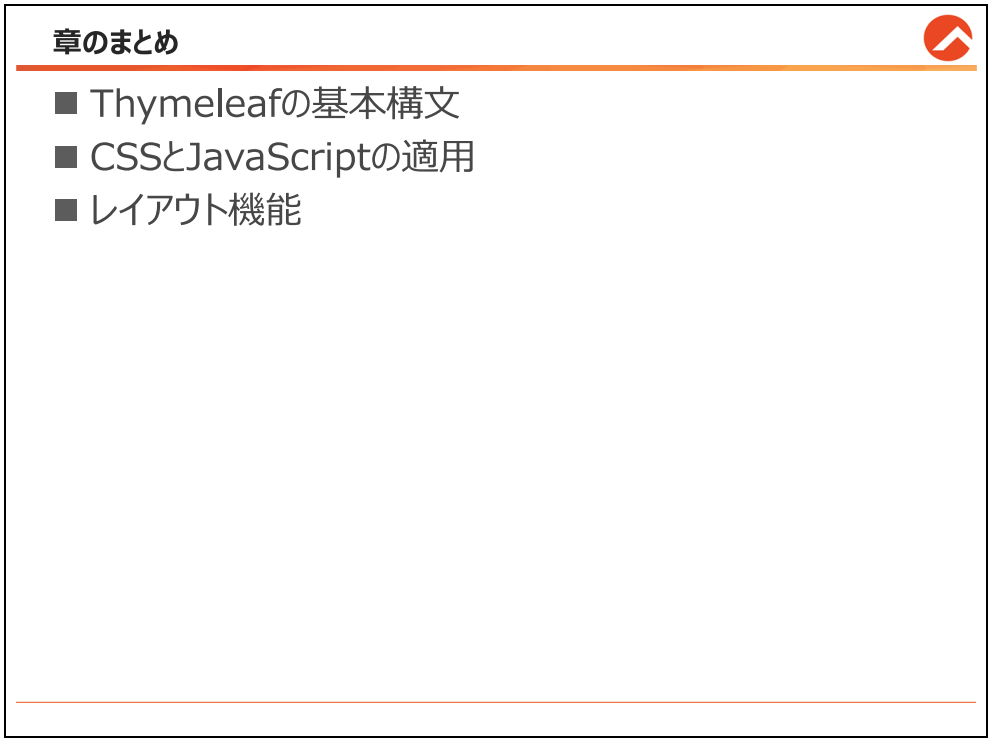
```
<html lang="ja" xmlns:th="http://www.thymeleaf.org"
      th:replace="~{layout2}">
<body>
    <main th:fragment="main_content">
        <h1>商品登録</h1>
```

HTML …/templates/fragment_sample/detail2.html

```
<html lang="ja" xmlns:th="http://www.thymeleaf.org"
      th:replace="~{layout2}">
<body>
    <main th:fragment="main_content">
        <h1>商品詳細</h1>
```

spring_answer_5 を起動し、**http://localhost:8080/product/fragment/list**を開きます。メニューから登録画面とLayout Dialect版の一覧、商品名から詳細画面を開けます。

章のまとめ



この章では、Spring Boot でシステムを実装する際に、追加で知っておくべき機能についてご説明しました。本書で全て紹介できなかったSpring の便利な機能もまだまだありますが、一番基本となる内容は網羅しましたので、これから皆様のSpring 開発知識の土台になると確信します。是非マスターしてください。